

Tools Engineering Design Manual

Governed Shared Calculations and Interchange Pathways

Release Status

This is the canonical editable QMD source for the Tools engineering design manual. It is a governance scaffold, not a released engineering manual. The file-level module inventory is complete for its declared tracked-source denominator; it is awaiting regeneration for the #5503 source additions and is not treated as current for this change. TOOLS-D4 registers one model-conditioned D-plane calculation and one swing/Rate-of-Closure textbook exemplar with exact source, consumer, test, fixture, equation, unit, and limitation traceability. The markerless-mocap exemplar remains explicitly blocked because its authority PR is unmerged and no markerless module exists on this exact source base. Release remains blocked by TOOLS-D5 through TOOLS-D9 and outstanding D4 approvals. Generated HTML, LaTeX, PDF, and DOCX artifacts are reproducible engineering records under TOOLS-D2, but none is approved for publication or engineering use.

Chapter 06 is an included, unregistered numerical-kernel note for triangle-multiset lamina moments. It is not a governed calculation or exemplar: the current freshness generator does not execute this pathway, and no source mesh/consumer admission or scientific review is supplied.

Documentation Authority and Release Boundary

Purpose

The manual will explain shared calculations and interchange contracts from theory and assumptions through typed inputs, units, coordinate and time conventions, algorithms, implementation symbols, tests, uncertainty, limitations, consumer surfaces, and artifacts. Tools source and evidence remain authoritative; the manual cannot approve a calculation that its owning source, evidence, license, or human review has not approved.

Canonical Source

Only QMD under `manuals/tools` is editable manual source. Existing user guides, ADRs, package documentation, generated API references, and publications are separate governed products. They can supply reviewed references, but none is a second mutable source for this manual.

Generated LaTeX, PDF, DOCX, and HTML are non-editable artifacts. A correction starts in canonical QMD, then regenerates every affected representation through the qualified toolchain. TOOLS-D2 owns that toolchain; TOOLS-D7 and TOOLS-D8 own artifact review and publication evidence.

Evidence and License Classes

Every calculation and claim must be classified as observed, derived, model-scenario, external, provisional, unavailable, or approved within an explicit scope. Every borrowed source and generated artifact must preserve its license and attribution. Content from `Tools_Private` or another non-public source may not enter this public manual without explicit public authority.

Approval and Failure Semantics

The default is deny-until-approved. Missing inventory coverage, source authority, units, coordinate or time convention, implementation symbol, test, validity range, uncertainty, license, artifact digest, page review, accessibility review, or human approval keeps the affected item blocked. Unsupported values are never replaced with synthetic or plausible values without an explicit model-scenario contract.

Exemptions

Freshness failures are release-blocking. The TOOLS-D1 module baseline is recomputed from tracked implementation and governed configuration files by `python -m scripts.build_tools_module_inventory --check`; later TOOLS-D6 work must extend freshness from the file baseline to stable calculation pathways. Future exemptions must be structured, narrowly scoped, justified, owned, reviewed, and expiring. An exemption cannot authorize an equation, scientific claim, artifact, operating use, license transfer, or public projection.

Current Boundary

TOOLS-D1 establishes a complete file-level denominator for its declared tracked scope. TOOLS-D2 establishes deterministic byte reproduction and governed semantic preservation across HTML, LaTeX, PDF, and DOCX. Its generated artifacts remain non-editable and unapproved. The conservative inventory does not establish stable calculation identity, equation correctness, validity ranges, uncertainty, page/accessibility approval, or publication authority. TOOLS-D4 registers one model-conditioned D-plane exemplar with explicit equation-to-code-to-test evidence while retaining review-required inventory authority. The markerless-mocap exemplar remains fail-closed because its source dependency is unmerged. Expansion, executable freshness, review, publication, and human approval remain blocked under TOOLS-D5 through TOOLS-D9.

Module and Calculation-Candidate Inventory

Purpose and Denominator

`manifests/module-inventory.json` is the deterministic TOOLS-D1 index for every tracked implementation source file and governed configuration module in the repository. It scans recognized Python, Python stub, JavaScript, TypeScript, Rust, MATLAB, C/C++, Arduino, shell, PowerShell, HTML, CSS, Prisma, JSON, YAML, TOML, INI, XML, URDF, and web-manifest files. Tests are evidence inputs rather than implementation rows. Generated, vendored, archived, experimental, example, documentation, resource, and manual trees are excluded explicitly in the manifest, so a consumer can reproduce the exact denominator. The index binds bounded entry shards by normalized SHA-256, contiguous index, entry count, and first/last path. Both the index and every shard remain below the repository's one-megabyte file budget; consumers reassemble one logical view only after verifying every shard and the exact declared shard set.

At this baseline, 3,439 modules are classified: 808 conservative calculation candidates and 2,631 non-calculation modules. One legacy JavaScript file has a non-UTF-8 encoding and remains explicitly blocked rather than being decoded by an implicit platform default. These counts describe inventory coverage only; they are not test coverage, calculation approval, or publication readiness.

Deterministic Identity and Integrity

Each file receives a provisional path-derived identifier and a SHA-256 digest of normalized text bytes. The normalization maps CRLF and bare CR to LF before hashing. For module path (p_i) and normalized content digest (h_i), the tree authority is

$$H_{\text{tree}} = \text{SHA256}(\|_{i=1}^n p_i : h_i),$$

where rows are ordered by normalized repository-relative path. This contract makes the result independent of Windows checkout line endings while retaining content changes. The path-derived identifier is not a stable calculation ID; TOOLS-D3 owns the structural chapter contract; TOOLS-D4 owns exemplar registration and stable equation-to-code identities that survive file movement.

Classification Contract

A module is a calculation candidate only when its normalized path contains a governed calculation marker, such as dynamics, kinematics, pressure drop, signal processing, geometry, solver, energy, force, or trajectory, or when the source imports a recognized scientific mathematics library. All other modules are non-calculation. This intentionally conservative static screen can have false positives and false negatives, so every calculation candidate has `authority_status=provisional` and `review_status=review-required`.

The inventory also records language, purpose where statically available, maintainer family, public surfaces and routes, mapped tests and validation, ADRs, equation references, literal units, citations, artifact digests, risk tags, normalized byte count, and content digest. Missing tests, sources, units, chapters, equation pathways, validation, routes, or artifact digests remain `unavailable`, `not-applicable`, or explicitly pending their owning subepic; they are never synthesized. Non-calculation status means only that the conservative detector found no calculation signal. It does not waive public-API, security, licensing, accessibility, or downstream compatibility review.

Consumer and Failure Contract

The repository-owned schema is `schemas/module-inventory.schema.json`, version `tools-module-inventory/1.0.0`. It extends the program-owned calculation registry without copying or competing with that shared schema. Both JSON Schema validation and the Python consumer reject unknown fields, unsupported versions, unsafe paths, invalid hashes, duplicate identifiers, unsorted duplicate traceability arrays, denominator mismatches, and unsupported state values.

Any tracked in-scope module addition, removal, rename, or normalized content change makes inventory check mode fail until the manifest is regenerated and reviewed. A green inventory check proves only deterministic classification and freshness of this file-level baseline. Public projection remains blocked until the remaining stable-pathway, page, accessibility, licensing, publication, and human-approval gates are complete.

Reproducible Rendering Contract

Outcome and Authority

TOOLS-D2 renders the canonical QMD into deterministic HTML, LaTeX, PDF, and DOCX representations. Generated outputs are engineering records, not approval. Only QMD may be corrected directly. Every output remains blocked from public projection until semantic, page, accessibility, provenance, licensing, and human-review gates owned by later subepics are complete.

The renderer uses the pinned command versions and inputs recorded in `toolchain-lock.json`. It verifies the exact first version line before render, expands only bounded QMD includes, uses the governed bibliography and reference DOCX, disables TeX shell escape, normalizes volatile package metadata, and binds every representation into `manifests/artifacts.json`. The command model follows the documented Pandoc conversion contract (Pandoc contributors 2026) while QMD remains compatible with the pinned Quarto authoring surface (Posit Software, PBC 2026).

Semantic, Unit, and Warning Preservation

The semantic contract is deliberately format-neutral. Every representation must retain the governed title, release boundary, representative unit string, warning, figure label, and bibliography identity before receiving a shared semantic digest. Example unit string: 10 m/s.

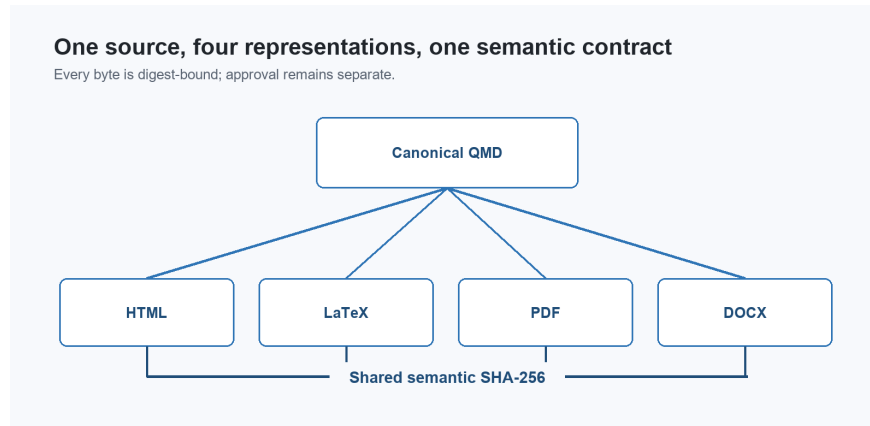


Figure 1: Deterministic rendering pathway.

WARNING: Provisional calculations remain blocked.

This gate proves preservation of required meaning across the four representations. It does not prove calculation correctness, equation coverage, accessibility, page aesthetics, or release approval. Missing text, a stale digest, an unpinned tool, an unsafe include, or an absent artifact fails closed.

Reproduction and Consumer Contract

Run `python -m scripts.render_tools_design_manual --verify-toolchain` before a qualified build. Run `python -m scripts.render_tools_design_manual --render` to replace renderer-owned artifacts, then `python -m scripts.render_tools_design_manual --check` to validate source, toolchain, byte, and semantic bindings. The strict JSON Schemas allow consumers to validate the lock and manifest without importing producer code.

Artifact rows expose media type, byte count, byte digest, and shared semantic digest. The manifest also preserves the owning epic, review owner, explicit blockers, and a null immutable source commit while this local stack is not a merged authority. A consumer must reject unknown fields, versions, formats, paths, hashes, status promotion, or semantic divergence.

PDF Extractor Compatibility

Live QA verifies the exact PDF byte digest and page count, ordered page identity, normalized non-whitespace character count, first-line prefix, fonts, bookmarks, images and annotations. Extracted line counts remain recorded diagnostics: an extractor version can change line wrapping while the PDF bytes and normalized text stay identical. Line counts do not participate in inventory equality. Empty-page and unresolved-reference checks still fail closed. Regression tests preserve equality under body rewrapping and verify the existing ledger with the CI extractor. No rendered artifact is edited by this policy.

References

The cited Pandoc and Quarto records are resolved from the governed bibliography declared by the pinned toolchain contract; an unresolved citation fails the qualified render.

Required Textbook Chapter Contract

Outcome and Authority

TOOLS-D3 defines the public structure that every calculation-level textbook chapter must satisfy before registration. The contract is structural and traceability-focused: passing it does not approve an equation, implementation, dataset, artifact, license, or engineering use. TOOLS-D4 now registers one verified-unapproved swing and Rate-of-Closure exemplar; missing markerless authority remains an explicit blocker rather than an inferred calculation.

Two version identifiers are fixed:

- Contract: `tools-textbook-chapter-contract/1.0.0`
- Registry: `tools-textbook-chapter-registry/1.0.0`

Their repository-owned JSON Schemas allow external consumers to validate shape without importing producer code. The Python consumer then applies stricter invariants for normalized paths, ordering, uniqueness, authority state, and QMD content.

Required Chapter Anatomy

Fourteen ordered sections are mandatory. A registered chapter must present, in order: Purpose and Scope; Design by Contract; Coordinates, Frames, and Time; Units and Dimensions; Derivation and Assumptions; Algorithm; Implementation Symbols; Error and Failure Semantics; Uncertainty and Sensitivity; Verification and Validation; Limits and Applicability; Worked Examples; References and Provenance; and Revision History.

The contract also declares mandatory third-level headings. For example, Design by Contract requires Preconditions, Postconditions, and Invariants. Coordinates requires frame, handedness, and time declarations. Verification and Validation requires test evidence and acceptance criteria. Exact ordered headings remove format-dependent inference and make an omitted obligation a deterministic failure.

Registration and Traceability

Each registered chapter has one stable chapter ID and one normalized QMD path. Its descriptor binds the chapter version and provisional authority to an owner, review owner, stable calculation IDs, module IDs, verification tests, source citations, explicit limitations, and revision history. All traceability arrays are non-empty, unique, and sorted; duplicate chapter IDs or paths are rejected.

An approved chapter claim is unsupported by this D3 contract because later subepics own page, accessibility, publication, and human-review evidence. D3 accepts only provisional and verified-unapproved chapter descriptors. The manual and every generated representation therefore remain generated-unapproved.

Linter Algorithm and Evidence

The linter loads the contract and registry through strict typed consumers, resolves each registered path beneath `manuals/tools/chapters`, decodes UTF-8, and rejects forbidden placeholder or private-source tokens. It extracts level-2 and level-3 headings, compares them with the exact ordered contract, and checks that every required section has governed body content.

For normalized chapter text T and ordered section blocks S_i , the linter records deterministic evidence as

$$h_T = \text{SHA256}(\text{LF}(T)), \quad h_i = \text{SHA256}(\text{LF}(S_i)).$$

These hashes detect byte-level content drift while remaining independent of Windows or POSIX checkout line endings. They are lint evidence, not scientific approval or an immutable publication digest.

Failure and Ownership Boundaries

Missing files, invalid UTF-8, unsafe paths, unknown fields or versions, duplicate identities, unsorted traceability, missing or reordered headings, empty governed content, placeholders, private-source references, unsupported status promotion, and stale generated artifacts all fail closed. The linter does not synthesize equations, units, sources, examples, tests, or limitations.

TOOLS-D4 owns exemplar chapter registration and its fail-closed dependency coverage. TOOLS-D5 expands phased module coverage, TOOLS-D6 owns executable freshness, TOOLS-D7 owns rendered and accessibility review, and TOOLS-D8 owns immutable public projection and human approval.

Consumer Workflow

Authors edit only canonical QMD and the machine-readable chapter registry. Run these commands in order:

1. `python -m scripts.lint_tools_textbook_chapters`
2. `python -m scripts.check_tools_exemplars`
3. `python -m scripts.render_tools_design_manual --render`

Swing and Rate-of-Closure D-Plane Geometry

Purpose and Scope

This governed exemplar explains how `swing_sim` computes frame-explicit three-dimensional D-plane geometry and how Rate of Closure supplies three distinct rigid-body velocity points to that calculation. It is for developers, scientific reviewers, and integrators who need to reproduce the equations and follow every value into code and tests. It covers geometric direction relationships only. It does not predict ball launch, spin magnitude, aerodynamic flight, player outcome, equipment quality, or an operating decision. The calculation and rendered artifacts are verified-unapproved.

Design by Contract

Preconditions

`analyze_dplane` accepts travel, face-normal, target, and up vectors with exactly three finite components. Face normal, target, and up must each have Euclidean norm greater than 10^{-12} . Target and up must be orthogonal to absolute dot product 10^{-10} . The Rate-of-Closure adapter additionally requires a typed `SimulationRun` with increasing sample times and internally consistent pose, twist, club, scenario, and inspection-event records.

Postconditions

Every available direction is normalized and every angle is in degrees. Spin loft is the principal angle on $[0, 180]$. A defined D-plane supplies a unit normal perpendicular to travel and face. Zero, parallel, antiparallel, and unprojectable states return explicit status or null fields; they never invent a plane, horizontal heading, or face-to-path result.

Invariants

A common proper rotation of travel, face, target, and up preserves spin loft, planar spin loft, face-to-path, and intrinsic tilt. Reflecting the right axis reverses signed horizontal relationships while preserving spin loft. Multiplying a nonzero travel vector by a positive scalar does not change the geometry because only direction enters the method.

Coordinates, Frames, and Time

Coordinate Frames

The default application frame is `app_frame:x_target,y_up,z_right`. Let $\hat{\mathbf{t}}$ be target and $\hat{\mathbf{u}}$ be up. The implementation constructs right as

$$\hat{\mathbf{r}} = \frac{\hat{\mathbf{t}} \times \hat{\mathbf{u}}}{\|\hat{\mathbf{t}} \times \hat{\mathbf{u}}\|}.$$

Rate of Closure interpolates the inspection-event rigid transform and twist, then expresses reference, face-center, and contact vectors in this same frame.

Axis and Handedness

Positive path and face headings rotate from target toward right. Elevation is positive toward up. Target-times-up fixes the sign of right; reversing the cross-product order would reverse every signed horizontal result. The D-plane normal is travel-times-face, so its orientation also depends on declared order.

Time Convention

The geometry is instantaneous and has no integration clock. Rate of Closure evaluates it at `inspection_time_s`. Pose and twist are interpolated to that event time; the closest sampled index is audit metadata and a bounded tangent fallback. A miss remains a closest-approach event and is not relabeled impact.

Units and Dimensions

Symbol and Unit Table

Symbol or field	Meaning	Unit
$\mathbf{v}$	reference, face-center, or contact velocity	m/s
$\hat{\mathbf{n}}$	delivered face normal	1
$\hat{\mathbf{t}}, \hat{\mathbf{u}}, \hat{\mathbf{r}}$	target, up, right axes	1
λ	three-dimensional spin loft	degree
p, a, f, ℓ	path, attack, face, and dynamic-loft angles	degree
$\hat{\mathbf{d}}$	D-plane normal	1
<code>event_time_s</code>	interpolation time	s

Dimensional Checks

Normalizing $\mathbf{v}$ divides m/s by m/s, producing a unit direction. Dot and cross products operate on dimensionless directions. `acos` and `atan2` return dimensionless radians, converted to degrees. The rigid-body equation $\mathbf{v}_P = \mathbf{v}_O + \boldsymbol{\omega} \times \mathbf{r}_{OP}$ is consistent: rad/s is treated as $1/s$, and multiplying by metres produces m/s.

Derivation and Assumptions

Assumptions

The calculation assumes Euclidean three-space, a right-handed orthonormal target frame, finite floating-point inputs, and one instantaneous delivered face normal. It treats the D-plane as geometry spanned by travel and face; collision dynamics and aerodynamic response lie outside this method. Rate of Closure assumes its interpolated homogeneous pose is rigid and its twist stores angular components followed by translational components.

Derivation

Normalize travel and face as $\hat{\mathbf{v}} = \mathbf{v}/\|\mathbf{v}\|$ and $\hat{\mathbf{n}} = \mathbf{n}/\|\mathbf{n}\|$. Their dot product is the cosine of the included angle. Clipping protects $\arccos$ from roundoff beyond the mathematical interval.

$$\lambda = \frac{180}{\pi} \cos^{-1}(\text{clip}(\hat{\mathbf{v}} \cdot \hat{\mathbf{n}}, -1, 1)).$$

When the vectors are not collinear, the oriented plane normal is

$$\hat{\mathbf{d}} = \frac{\hat{\mathbf{v}} \times \hat{\mathbf{n}}}{\|\hat{\mathbf{v}} \times \hat{\mathbf{n}}\|}.$$

For any direction $\mathbf{q}$, remove its vertical component: $\mathbf{q}_h = \mathbf{q} - (\mathbf{q} \cdot \hat{\mathbf{u}})\hat{\mathbf{u}}$. Heading and elevation are $\text{atan2}(\mathbf{q}_h \cdot \hat{\mathbf{r}}, \mathbf{q}_h \cdot \hat{\mathbf{t}})$ and $\text{atan2}(\mathbf{q} \cdot \hat{\mathbf{u}}, \|\mathbf{q}_h\|)$. Face-to-path wraps $f - p$ to $[-180, 180)$. Rate of Closure obtains face-center velocity from rigid-body kinematics before applying the same geometry.

Algorithm

Inputs

The standalone calculation consumes `travel_vector`, `face_normal`, optional `target` and `up`, and descriptive `frame_id`. The consumer accepts one immutable `SimulationRun` and derives inspection pose, twist, shaft geometry, contact lever, face normal, leading-edge tangent, and arc tangent rather than asking callers to assemble those values.

Procedure

1. Convert each vector to a finite three-component array and validate the frame.
2. Normalize face, target, and up; derive right from target-times-up.
3. Compute face heading and dynamic loft even if travel is zero.
4. Return `zero_travel` when travel norm is at most 10^{-12} .
5. Normalize travel; compute path, attack, face-to-path, and spin loft.
6. Return `parallel` or `antiparallel` for a singular cross product.
7. Otherwise derive plane normal, tilt, inclination, and ground intersection.
8. Rate of Closure repeats the calculation for reference, face center, and contact so a consumer cannot silently confuse those datums.

Standalone work is constant time and memory. Sector rendering is linear in the caller-bounded segment count and returns `segments + 1` unit directions.

Outputs

`DPlaneAnalysis` returns status, frame, normalized basis vectors, optional plane and ground-intersection directions, three-dimensional and planar spin loft, residual, path, attack, face, dynamic loft, face-to-path, azimuth, tilt, and inclination. `ImpactKinematicSnapshot` wraps three analyses with event, geometry basis, limitations, pose, and wedge-kinematics evidence.

Implementation Symbols

Public Symbols

The stable `shared.python.swing_sim.impact` facade exports `DPlaneStatus`, `DPlaneAnalysis`, `analyze_dplane`, and `spin_loft_sector_directions`. Rate of Closure exports `ImpactKinematicSnapshot` and `impact_kinematics_for_run`. Consumers should not import private helpers such as `_heading_deg` or `_shaft_geometry`.

Source Mapping

Calculation `TOOLS-DPLANE-GEOMETRY` maps spin loft, normal, heading, elevation, singular states, and sectors to `src/shared/python/swing_sim/impact/dplane.py`. The Rate-of-Closure datum and point-velocity mapping lives in `src/rate_of_closure/simulation/impact_kinematics.py`. Both are pinned to D3 base `916fb9f27f486486f0c071dcc6f15f81f68ecb18`; their provisional module IDs and public symbols remain recorded in the module inventory.

Error and Failure Semantics

Error Conditions

Wrong vector shape, non-finite values, zero face/target/up, or a nonorthogonal frame raises `ValueError`. A non-`SimulationRun` input raises `TypeError`. Nonincreasing times, zero shaft geometry, zero path fallback, or degenerate projected leading edge also fail preconditions. These failures are distinct from valid singular geometry.

Recovery and Cancellation

Callers recover by correcting source data or frame; the method does not coerce, interpolate missing vectors, or substitute a default. Valid singular statuses may be displayed with null fields intact. The synchronous calculation is bounded and has no external I/O, so it has no mid-operation cancellation seam. Higher-level cancellation occurs before calls or between complete runs.

Uncertainty and Sensitivity

Uncertainty Sources

The implementation does not estimate covariance. Uncertainty can enter through travel, face orientation, coordinate transform, event interpolation, club geometry, or the choice of reference versus contact datum. Floating-point clipping addresses numerical roundoff only; it is not a measurement uncertainty model. Angles therefore remain model-conditioned rather than observed truth.

Sensitivity

Small perturbations are smooth away from singular states. Sensitivity rises when horizontal projection approaches zero because heading becomes undefined, and when travel and face approach parallel or antiparallel because a small cross product cannot define a stable normal. The API exposes these limits as null or typed status; downstream code must not carry forward a prior frame's plane.

Verification and Validation

Test Evidence

`test_matches_cross_client_golden_contract` executes three analytical cases from `dplane_golden_v1.json` at absolute tolerance 10^{-9} degree. Singular-state tests cover zero travel, parallel, antiparallel, vertical projection, and invalid frames. Rotation and reflection tests pin invariants. The Rate-of-Closure integration test derives face-center velocity independently as $\mathbf{v}_O + \boldsymbol{\omega} \times \mathbf{r}$, then proves the consumer normalizes that value for its face-center D-plane.

Acceptance Criteria

The exemplar passes only when strict JSON Schema and typed loader agree; calculation, chapter, module, source, consumer, test, and golden-fixture links all resolve; the fourteen-section linter and focused tests pass; and HTML, LaTeX, PDF, and DOCX share the governed semantic digest. This evidence verifies reproduction, not scientific or publication approval.

Limits and Applicability

Validity Range

Finite vectors may have any representable scale, but face and frame axes must exceed norm 10^{-12} , and target/up orthogonality must satisfy 10^{-10} . Spin loft spans 0 through 180 degrees. Signed relationships wrap to $[-180, 180)$. The golden fixture qualifies only its declared cases and 10^{-9} -degree tolerance.

Known Limitations

The method does not compute ball response, impulse, spin rate, lift, drag, landing, biomechanics, or camera-derived pose. It does not establish that an external device's axes, timestamps, or face normal match this contract. Rate of Closure downstream flight currently consumes reference-point translation; shaft-induced contact velocity remains analysis. The markerless-mocap exemplar is deliberately absent because TOOLS-M0 issue #4708 and PR #4734 are open and no markerless module exists on this D3 base. Unmerged content is not promoted.

Worked Examples

Example Inputs

The governed square-descending fixture supplies $\mathbf{v} = (0.9986295347545738, -0.052335956242943835, 0)$, with path 0° and attack -3° . Its face normal is $\mathbf{n} = (0.9781476007338057, 0.20791169081775934, 0)$, with face 0° and dynamic loft 12° . Target and up use default unit axes. The fixture tolerance is 10^{-9} degree.

Example Results

The dot product is $\cos 15^\circ$, so spin loft is 15° . Dynamic loft minus attack is $12 - (-3) = 15^\circ$, making planar residual zero. Travel-times-face points along right, giving $\hat{\mathbf{d}} = (0, 0, 1)$, tilt 0° , inclination 90° , and ground intersection at 0° . Status is `defined`. Values come from the cross-client golden fixture and executable Python test, not renderer-only text.

References and Provenance

References

Primary implementation authority is exact source at 916fb9f27f486486f0c071dcc6f15f81f68ecb18. Numerical expectations come from `dplane-golden-v1`, with independent behavior exercised by the tests in the registry. External scientific literature has not completed review, so this chapter makes no literature-backed ball-flight or equipment-performance claim.

Provenance

The calculation registry records method version 1.0.0, symbols and commit, source digest, inputs, units, contracts, equation anchors, tests, artifacts, limitations, and approval blockers. D4 coverage separately records markerless as blocked. Generated HTML, LaTeX, PDF, and DOCX derive from canonical QMD via the pinned D2 toolchain and remain generated-unapproved.

Revision History

Revision Record

Version 1.0.0, dated 2026-08-26, registers the first Tools textbook exemplar: frame-explicit D-plane geometry shared by swing simulation and Rate of Closure, with executable golden values and fail-closed markerless dependency status. Later revisions must update the descriptor, method version when mathematics or behavior changes, tests, provenance, limitations, and every generated artifact.

Opaque Native Restart and Saved-Input Replay

Scope and Authority

Tools #5470 supplies typed transport for native state that cannot be represented truthfully as a named numeric state vector. The wire format is `native-state-replay/1.1.0`. Its complete initial-state authority is a native artifact descriptor and separately resolved immutable payload bytes. Numeric observations are deliberately absent from this restart authority. The existing `experiment-replay/1.0.0` numeric reader and semantics remain unchanged; that reader rejects the native envelope instead of discarding hidden state.

These are interchange contracts, not a physics solver or qualification system. UpstreamDrift #11921 owns actual R2025b decoding, independent integration, model coverage and acceptance. All required engine/model rows remain visible.

State, Model and Runtime Identity

`NativeStateArtifact` binds format/version and expected decoded native class, exact producing runtime/provider, source and loaded-model SHA-256 values, native clock, solver and effective configuration/checksum policy, byte size, payload digest and explicit complete-native-restart or observables-only role. `NativeReplayModel` preserves inventory model/variant identity and ordered input mapping separately. Native artifacts cannot substitute an observables-only scope during replay admission.

`freeze_native_state_artifact` verifies an owned immutable byte snapshot. It rejects wrong size/hash and numeric/list coercion; it never deserializes or executes the payload. A consumer must load these verified bytes and independently verify actual decoded class, time, runtime and model compatibility. Hashing MAT bytes identifies one serialization; it does not establish canonical semantic state encoding, source-to-binary equivalence or transitive library closure.

Native Clock and Input Boundary

The saved native `snapshot_time_seconds` and `start_time_seconds` are finite and ordered, with an explicit native clock identity. Saved input histories reuse the existing interval-relative time and interpolation contracts. Native replay time is the saved snapshot plus each relative input time; the operating point is never retimed to zero. Overflow or loss of strict ordering at native floating point precision is rejected. Requested intervals and an explicitly supplied solver step must also survive translation to the native epoch within the larger of 1e-9 relative interval error or 64 ULPs of that interval. Epoch ULPs never relax this policy. Consumers must check effective native solver precision too. Physical input units, muscle excitation versus activation, force versus command, and explicit contact identity remain governed by the shared `InputHistory` and `ReplayExecutionPolicy` contracts.

Integrity and Failure Semantics

`NativeReplayIntegrity` retains shared component hashes and adds an entire envelope digest, including experiment identity. Strict JSON rejects duplicate or unknown fields, unsupported versions and stale integrity. Capability availability remains distinct from declared support; required unavailable or unsupported capabilities remain blockers. Metadata equality does not prove a native engine followed the declared policy.

Native Consumer Requirements

For R2025b, preserve hidden Simscape/block state, solver/execution state and operating-point compatibility. Disable block reduction before capture and bind effective solver settings rather than unresolved UI values such as `auto`. Verify a nonzero-time saved/reloaded restart against uninterrupted native integration, including discrete/hidden state. Preserve wrong model/runtime/time, tampered or missing blob, stale artifact and observable-downgrade negatives. No state reset is permitted during the requested saved-input replay horizon.

Evidence and Remaining Work

Initial missing-module, coercion, public-facade, duplicate-key and experiment identity failures were observed before fixes. The current focused suite passes 77 tests, including 23 unchanged numeric T01 tests and 11 native JSON Schema tests. Astra review supplied reproduced string-channel coercion and large-epoch interval-retiming negatives; both failed before correction. Configured source typing passes. The schema reuses stable numeric-contract definitions by local reference. The manual renderer uses the pinned toolchain; QA inspects every actual page and rejects missing, duplicate or reordered page records as the manual grows. Inventory/governance/normal hooks and actual R2025b integration remain pending at this development checkpoint. This chapter does not approve a scientific calculation or manual release.

Primary References

MathWorks R2025b ModelOperatingPoint defines native restart state and the read-only snapshot clock; its numeric logged-state view excludes internal Simscape state. MathWorks R2025b Operating-Point Workflow documents compatibility and effective configuration constraints. These native claims require separate actual execution evidence from UpstreamDrift.

Triangle-Multiset Lamina Mass Moments

Purpose and Scope

This note describes the numerical kernel introduced for Tools #5503. Given triangles already expressed in one orthonormal metre frame and one positive total mass, it computes the center of mass and inertia tensor of a zero-thickness lamina whose surface-mass density is proportional to unsigned triangle area. The source is a triangle **multiset**: repeated faces count repeatedly. The kernel is included in the manual for transparent review but is not yet a registered or approved calculation. It does not load or admit a geometry asset, establish frame authority, infer tissue ownership, model volume or thickness, or qualify a native humanoid/body model.

Design by Contract

Preconditions

`vertices` must convert to a finite `float64` array of shape $(N, 3)$. `faces` is an integer array of shape $(T, 3)$ whose indices lie in $[0, N)$. `mass` is a positive, finite, non-boolean scalar in kilograms. All vertex coordinates are metres in one caller-declared orthonormal coordinate frame. At least one input triangle must have positive representable area; each nonzero area and its normalized fraction must be representable as a positive `float64`, and total area must remain finite.

Postconditions

The result reports the area-weighted center in metres and the center-of-mass inertia tensor in kg m^2 for the declared lamina. It also reports total unsigned area, zero-area face count, exact coincident positive-area face count, and indexed boundary/nonmanifold edge counts. The duplicate diagnostic counts each repeated coordinate face beyond its first occurrence. The compatible `InertiaResult` uses `volume=0`, `was_watertight=False`, and the distinct `triangle_multiset_lamina_mass` mode so it cannot imply a closed solid.

Invariants

Reordering faces, translating the complete geometry, or applying a proper rigid rotation preserves mass and scalar invariants; translation changes the center by the same translation and does not change central inertia. Scaling coordinates by a positive factor scales area by its square, preserves the area fractions, and scales inertia by the square when total mass is held fixed. Subdivision that preserves the geometric surface and area measure

preserves the moments within numerical tolerance. A repeated positive-area face retains its multiplicity in both mass allocation and the diagnostic count.

Coordinates, Frames, and Time

Coordinate Frames

The function accepts coordinates in the caller's existing frame and performs no frame transform. All vertices must use the same origin, axes, handedness, and scale. No source-frame identifier is present in this numerical API; a consumer that needs provenance or a native body frame must bind that separately before calling it.

Axis and Handedness

The covariance and inertia formulas use Euclidean three-dimensional vectors. They are invariant to proper orthogonal basis rotation when both inputs and outputs are transformed consistently. Triangle winding does not select a normal or sign: area is unsigned and the represented surface has no orientation semantics.

Time Convention

This is a static geometry calculation and has no time, interpolation, or integration policy. It cannot be used to infer a time-varying body shape.

Units and Dimensions

Symbol and Unit Table

Symbol or field	Meaning	Unit
$\mathbf{x}$, triangle vertices	Position in the common input frame	m
A_i , A	Triangle area and total multiset area	m ²
w_i	Triangle area fraction	1
m	Caller-specified total lamina mass	kg
$\mathbf{c}$	Area-weighted center of mass	m
$\mathbf{C}$	Mass-normalized central covariance	m ²
$\mathbf{I}$	Central inertia tensor	kg m ²

Dimensional Checks

Area fractions are dimensionless. Nonzero areas and normalized face fractions that underflow to zero in `float64` are rejected, not assigned zero mass or reported as degenerate. No geometric area epsilon is used. The covariance has units m² because it is an area-weighted second central moment divided by total area. Multiplication by mass yields kg m². The zero-thickness convention sets reported volume to 0 m³; it is not an estimated enclosed volume.

Derivation and Assumptions

Assumptions

Each triangle carries uniform surface density, with one shared areal density $\sigma = m/A$ over the entire input multiset. Mass is apportioned by unsigned area. Triangles may be disconnected, overlap, coincide, or have inconsistent winding; overlap is not removed. Degenerate zero-area triangles contribute no mass and are reported separately. The derivation assumes ordinary Euclidean coordinates and no thickness or volumetric contribution.

Derivation

For triangle vertices $\mathbf{v}_{i1}, \mathbf{v}_{i2}, \mathbf{v}_{i3}$,

$$A_i = \frac{1}{2} \|(\mathbf{v}_{i2} - \mathbf{v}_{i1}) \times (\mathbf{v}_{i3} - \mathbf{v}_{i1})\|, \quad A = \sum_i A_i, \quad w_i = \frac{A_i}{A}.$$

The triangle centroid is $\mathbf{c}_i = (\mathbf{v}_{i1} + \mathbf{v}_{i2} + \mathbf{v}_{i3})/3$, and the lamina center is $\mathbf{c} = \sum_i w_i \mathbf{c}_i$. Uniform barycentric coordinates on one triangle have covariance

$$\mathbf{C}_i = \frac{1}{12} \sum_{k=1}^3 (\mathbf{v}_{ik} - \mathbf{c}_i)(\mathbf{v}_{ik} - \mathbf{c}_i)^T.$$

Applying the law of total covariance over triangle identity and within-triangle position gives

$$\mathbf{C} = \sum_i w_i [\mathbf{C}_i + (\mathbf{c}_i - \mathbf{c})(\mathbf{c}_i - \mathbf{c})^T].$$

For a mass distribution with central covariance $\mathbf{C}$,

$$\mathbf{I} = m [\text{tr}(\mathbf{C})\mathbf{1}_3 - \mathbf{C}].$$

This formula returns the standard inertia tensor, including signed off-diagonal tensor components. `InertiaResult.as_urdf_dict()` exports those same symmetric tensor entries as URDF `ixy`, `ixz`, and `iyz` coefficients. The exporter does not negate them: an actual MuJoCo URDF loadback test checks the reconstructed native tensor and kinetic energy for an asymmetric inertia. This verifies that particular exporter/consumer path, not every URDF parser or native engine.

Algorithm

Inputs

The public function `compute_triangle_lamina(vertices, faces, mass)` consumes only the arrays and scalar above. It does not consume a path, mesh object, transform, frame label, density, thickness, or native model.

Procedure

1. Validate the mass and array shapes, finiteness, face dtype, and index bounds.
2. Gather indexed triangles and compute unsigned area weights.
3. Reject nonpositive or nonfinite total area; omit only exactly zero-area triangles from moment/topology accumulation.
4. Choose the first vertex of a maximum-area face as a local origin, then compute weighted center and central covariance in that translated frame.
5. Convert covariance to inertia and reject nonfinite center or tensor values.
6. Count exact duplicate positive-area coordinate triangles independent of winding, and count edge degree using source vertex indices.
7. Return a distinct lamina mode and explicit diagnostics without repairing or modifying source arrays.

The numerical arrays are vectorized over faces. Working memory is linear in the number of input faces; uniqueness diagnostics sort/compare face and edge records and are not a streaming algorithm.

Outputs

`TriangleLaminaResult` contains an `InertiaResult`, total area, zero-area face count, duplicate positive-area coordinate-face count, indexed boundary-edge count, and indexed nonmanifold-edge count. The embedded legacy result object remains mutable for compatibility. Counts are diagnostics, not mesh validation.

Implementation Symbols

Public Symbols

`compute_triangle_lamina` and `TriangleLaminaResult` are defined in `src/shared/python/humanoid_character_builder/InertiaMode.TRIANGLE_MULTISSET_LAMINA_MASS` and the compatible `InertiaResult` representation are defined in `src/shared/python/humanoid_character_builder/mesh/inertia_calculator.py`. This note does not assert that the module is a stable published consumer API.

Source Mapping

The method maps to `_area_weights`, `_centered_moments`, and `compute_triangle_lamina` in `triangle_lamina.py`; mode identity is mapped in `inertia_calculator.py`. Analytic and metamorphic checks are in `src/shared/python/humanoid_character_builder/mesh/inertia_calculator.py`. This kernel is separate from `MeshInertiaCalculator`, which derives solid properties from volume-mesh processing and watertightness policy.

Error and Failure Semantics

Error Conditions

Wrong scalar kind or noninteger face indices raise `TypeError`. Invalid scalar range, malformed/nonfinite vertices, face shape or bounds, no positive finite total area, and nonfinite output moments raise `ValueError`. The function does not repair malformed topology or substitute default mass/geometry.

Recovery and Cancellation

The caller must correct source inputs and resubmit. There is no partial result, fallback solid calculation, or cancellation callback. Large inputs are bounded only by caller-provided memory and face count; this API provides no resource budget or streaming cancellation guarantee.

Uncertainty and Sensitivity

Uncertainty Sources

This kernel propagates no uncertainty distribution. Its values depend on caller-selected coordinates, mesh sampling, face multiplicity, declared mass, and the unverified assumption of uniform area density. It does not determine whether those assumptions are appropriate for an object or body segment.

Sensitivity

Area and second moments are sensitive to coordinate perturbations and changes in tessellation or face multiplicity. Very small representable positive areas remain in the calculation; no tolerance silently removes them. Nonzero areas or fractions not representable as positive `float64` are rejected. Summation and floating-point rounding still affect results. The local-origin accumulation reduces avoidable cancellation for translated geometry but does not provide an error bound for every scale or conditioning regime.

Verification and Validation

Test Evidence

`test_triangle_analytic_mass_com_and_off_diagonal` checks the exact triangular lamina result. Other tests check a rectangular plate and subdivision, an analytic cube shell, rigid transforms/scaling/input immutability, duplicate and degenerate faces, remote negligible-area faces, small representable and unrepresentable areas, complex-coordinate rejection, and invalid input classes. These are independent synthetic analytic or metamorphic tests; they are not validation against a measured object or biomechanical segment.

Acceptance Criteria

Numerical kernel acceptance requires finite typed inputs, correct analytic moments, expected rigid-transform and subdivision invariants, explicit degeneracy diagnostics, and fail-closed invalid-input behavior. Those criteria do not establish source asset provenance, unit/frame correctness of a producer, partial-overlap handling, biological mass distribution, or native-model admission. The calculation freshness generator currently executes only the D-plane pathway; this note therefore remains outside the registered calculation set and cannot be represented as freshness-verified or exemplar-verified.

Limits and Applicability

Validity Range

The equations apply to a finite positive-area triangle multiset in one orthonormal Euclidean metre frame with a positive finite caller-owned total mass. Degenerate faces may be present but contribute no area or mass. No evidence here establishes useful geometric scales beyond representable, finite floating-point inputs.

Known Limitations

Coincident triangles retain multiplicity; partial overlap is neither detected nor corrected. Duplicate detection is exact-coordinate and ignores winding, whereas edge topology uses vertex indices. Indexed boundary/nonmanifold counts do not prove surface closure, manifoldness, or self-intersection absence. No volume, thickness, density field, material, anatomy, frame transform, native asset loader, downstream consumer or physical acceptance is included. The registered calculation, freshness workflow, source asset producer and any engineering-use approval remain blocked.

Worked Examples

Example Inputs

Use vertices $(0, 0, 0)$, $(3, 0, 0)$, and $(0, 6, 0)$ metres, one face with indices $(0, 1, 2)$, and total mass 2 kg. The triangle area is 9 m^2 and the face receives all of the declared mass.

Example Results

The centroid is $(1, 2, 0) \text{ m}$. Integrating the uniform triangular area gives $\mathbf{C} = \begin{bmatrix} 1/2 & -1/2 & 0 \\ -1/2 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} \text{ m}^2$. The returned

inertia matrix is $\mathbf{I} = \begin{bmatrix} 4 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 5 \end{bmatrix} \text{ kg m}^2$, with all three indexed edges counted as boundary edges. The example

verifies the kernel's declared lamina equations; it is not an asset-backed exemplar or evidence about a physical segment.

References and Provenance

References

- `src/shared/python/humanoid_character_builder/mesh/triangle_lamina.py` — implementation of the triangle-multiset weighting and moments.
- `src/shared/python/humanoid_character_builder/mesh/inertia_calculator.py` — compatible result/mode definitions and separate solid-mesh pathway.
- `src/shared/python/humanoid_character_builder/tests/test_triangle_lamina.py` — analytic and metamorphic synthetic tests.
- `src/shared/python/humanoid_character_builder/tests/test_urdf_inertia_export.py` — symmetric-entry round-trip and an actual MuJoCo tensor/kinetic-energy readback.
- Inertia tensor of a triangle in barycentric coordinates — independent derivation reference for triangle area moments.
- ROS 2 KDL URDF parser — primary consumer source for the direct tensor-coefficient interpretation tested by the exporter regression.

Provenance

This chapter is a provisional technical note for Tools #5503/#5504. The exact source and tests are tracked in the same development change. The canonical module inventory is regenerated, but executable freshness support beyond the D-plane calculation, governed calculation registration, source geometry producer, anatomical consumer traceability, external scientific review, artifact/page review, and human approval are not complete. The MuJoCo check is limited to an analytic asymmetric URDF fixture and does not establish all-engine parity. No private mesh or capture data is used or reproduced here.

Revision History

Revision Record

- 0.1.0 (2026-10-10): Document the provisional triangle-multiset lamina kernel and its analytic/synthetic evidence; keep governed registration and all source/consumer/engineering qualification blocked.

Pandoc contributors. 2026. *Pandoc User's Guide*. Pandoc. <https://pandoc.org/MANUAL.html>.

Posit Software, PBC. 2026. *Quarto Guide*. Posit Software, PBC. <https://quarto.org/docs/guide/>.